

Multimodal Visual Voice Interaction

Multimodal Visual Voice Interaction

1. Concept Introduction
 - 1.1 What is "Visual Understanding"?
 - 1.2 Implementation Principles Overview
2. Project Architecture
 - Key Code
 1. Tool Layer Entry (`targetmodel/utils/tools_manager.py`)
 2. Model Interface Layer (`targetmodel/utils/large_model_interface.py`)
 - Code Analysis
3. Practical Operations
 - 3.1 Configure Online LLM
 - 3.2 Start and Test the Function

1. Concept Introduction

1.1 What is "Visual Understanding"?

In the `targetmodel` project, the **Multimodal Visual Understanding** function refers to enabling the robot to not only "see" a pixel matrix but also truly "understand" the content, objects, scenes, and relationships between them in an image. This is like giving the robot a pair of eyes that can think.

The core tool of this function is `seewhat`. When the user gives an instruction like "look at what's here", the system calls this tool, triggering a series of background operations, and finally feeds back the AI's analysis of the real-time camera view to the user in natural language.

1.2 Implementation Principles Overview

The basic principle is to input two different types of information—**images (visual information)** and **text (language information)**—into a powerful multimodal large model (such as LLaVA).

1. **Image Encoding:** The model first converts the input image into numerical vectors that a computer can understand through a Vision Encoder. These vectors capture the color, shape, texture, and other features of the image.
2. **Text Encoding:** At the same time, the user's question (such as "what's on the table?") is also converted into a text vector.
3. **Cross-modal Fusion:** The most critical step, the model fuses image vectors and text vectors in a special "Attention Layer". The model learns to "attend" to parts of the image related to the question. For example, when asked about "table", the model will pay more attention to areas in the image that match "table" features.
4. **Generate Answer:** Finally, a Large Language Model (LLM) component generates a descriptive text answer based on the fused information.

Simply put, it is **using text to "illuminate" the corresponding parts of the image, and then describing the "illuminated" parts in language.**

2. Project Architecture

Key Code

1. Tool Layer Entry (`largemodel/utils/tools_manager.py`)

The `seewhat` function in this file defines the execution flow of this tool.

```
# From largemodel/utils/tools_manager.py

class ToolsManager:
    # ...

    def seewhat(self):
        """
        Capture camera frame and analyze environment with AI model.

        :return: Dictionary with scene description and image path, or None if failed.
        """
        self.node.get_logger().info("Executing seewhat() tool")
        image_path = self.capture_frame()
        if image_path:
            # Use isolated context for image analysis
            analysis_text = self._get_actual_scene_description(image_path)

            # Return structured data for the tool chain
            return {
                "description": analysis_text,
                "image_path": image_path
            }
        else:
            # ... (Error handling)
            return None

    def _get_actual_scene_description(self, image_path, message_context=None):
        """
        Get AI-generated scene description for captured image.

        :param image_path: Path to captured image file.
        :return: Plain text description of scene.
        """
        try:
            # ... (Build Prompt)

            # Force use of a plain text system prompt with a clean, one-time context
            simple_context = [{
                "role": "system",
                "content": "You are an image description assistant. ..."
            }]

            result = self.node.model_client.infer_with_image(image_path, scene_prompt,
message=simple_context)
```

```

        # ... (Process result)
        return description
    except Exception as e:
        # ...

```

2. Model Interface Layer (`largemodel/utils/large_model_interface.py`)

The `infer_with_image` function in this file is the unified entry point for all image understanding tasks. It is responsible for calling specific model implementations based on configuration.

```

# From largemodel/utils/large_model_interface.py

class model_interface:
    # ...
    def infer_with_image(self, image_path, text=None, message=None):
        """Unified image inference interface"""
        # ... (Prepare message)
        try:
            # Determine which specific implementation to call based on self.llm_platform
            if self.llm_platform == 'ollama':
                response_content = self.ollama_infer(self.messages, image_path=image_path)
            elif self.llm_platform == 'tongyi':
                # ... Call Tongyi model logic
                pass
            # ... (Other platform logic)
        # ...
        return {'response': response_content, 'messages': self.messages.copy()}

```

Code Analysis

The implementation of this function involves two main layers: the tool layer defines business logic, and the model interface layer is responsible for communication with large language models. This layered design is key to achieving platform generality.

1. Tool Layer (`tools_manager.py`):

- The `seewhat` function is the business core of the visual understanding function. It encapsulates the complete flow of the "look" action: first calling the `capture_frame` method to get an image, then calling `_get_actual_scene_description` to prepare an instruction (Prompt) for requesting the model to analyze the image.
- The most critical step is that it calls the `infer_with_image` method of the model interface layer. It doesn't care which model is used underneath; it only passes the two core pieces of data—"image" and "analysis instruction"—over.
- Finally, it packages the analysis result (plain text description) received from the model interface layer into a structured dictionary and returns it. This allows upper-layer applications to easily use the analysis results.

2. Model Interface Layer (`large_model_interface.py`):

- The `infer_with_image` function acts as a "dispatch center". Its main responsibility is to check the current platform configuration (`self.llm_platform`) and dispatch the task to the corresponding specific processing function (such as `ollama_infer` or `tongyi_infer`) based on the configuration value.
- This layer is key to adapting to different AI platforms. All platform-specific operations (such as data encoding, API call formats, etc.) are encapsulated in their respective processing functions.
- Through this approach, the business logic code in `tools_manager.py` can work without any modifications to support a variety of different backend large model services. It only needs to interact with the unified, stable interface of `infer_with_image`.

In summary, the execution flow of the `seewhat` tool reflects a clear responsibility separation pattern: `ToolsManager` is responsible for defining "what to do" (get image and request analysis), while `model_interface` is responsible for defining "how to do it" (based on current configuration, select the appropriate model platform and interact with it). This makes the tutorial's analysis universal—regardless of whether the user is in online or offline mode, the core code logic is consistent.

3. Practical Operations

3.1 Configure Online LLM

1. Update the key in the configuration file, open the model interface configuration file

`large_model_interface.yaml`:

```
vim ~/yahboomcar_ws/src/largemodel/config/large_model_interface.yaml
```

2. Fill in your API Key:

Find the corresponding section and paste the API Key you just copied. Here taking Qwen configuration as an example

```
# large_model_interface.yaml

## Qwen
qianwen_api_key: "sk-xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx" # Paste your Key
qianwen_model: "qwen-vl-max-latest" # You can choose models as needed, such as qwen-
turbo, qwen-plus
```

3. Open the main configuration file `yahboom.yaml`:

```
vim ~/yahboomcar_ws/src/largemodel/config/yahboom.yaml
```

4. Select the online platform to use:

Modify the `llm_platform` parameter to the platform name you want to use

```
# yahboom.yaml

model_service:
  ros_parameters:
    # ...
    llm_platform: 'tongyi' # Optional platforms: 'tongyi', 'spark', 'qianfan',
    'openrouter'
```

5. When using the voice module for broadcast, check if a device is occupying the voice module. You can first enter

```
sunrise@ubuntu:~/Desktop$ arecord -l
**** List of CAPTURE Hardware Devices ****
card 0: duplexaudioi2s1 [duplex-audio-i2s1], device 0: i2s1-voicehat-hifi voicehat-hifi-0 [i2s1-voicehat-hifi voicehat-hifi-0]
Subdevices: 0/1
Subdevice #0: subdevice #0
card 1: duplexaudio [duplex-audio], device 0: i2s0-(null) ES8326 HiFi-0 [i2s0-(null) ES8326 HiFi-0]
Subdevices: 1/1
Subdevice #0: subdevice #0
```

If the output is as shown in the image above, you can directly use the broadcast device number without modification. If different, open the configuration file

```
vim ~/yahboomcar_ws/src/largemodel/largemodel/model_service.py
```

Find the location in the image and modify it to plughw:1,0 or hw:1,0, then save.

```
472         except Exception as e:
473             self.get_logger().error(f"❌ Error in TTS callback: {e}")
474             # 出错时恢复麦克风
475             self.publish_wakeup_signal(False)
476
477     def play_tts_audio(self):
478         """播放TTS音频文件"""
479         try:
480             if not os.path.exists(self.tts_out_path):
481                 self.get_logger().error(f"❌ Audio file not found: {self.tts_out_path}")
482                 return
483
484             # 使用aplay命令播放
485             self.get_logger().info(f"🔊 Playing audio: aplay -D plughw:0,0 {self.tts_out_path}")
486
487             result = subprocess.run(
488                 ["mpg123", "-a", "plughw:1,0", self.tts_out_path],
489                 capture_output=True,
490                 text=False
491             )
492
493             if result.returncode == 0:
494                 self.get_logger().info("✅ TTS playback completed successfully")
495             else:
496                 self.get_logger().error(f"❌ TTS playback failed: {result.stderr}")
497
498         except Exception as e:
499             self.get_logger().error(f"❌ Error playing TTS audio: {e}")
500
```

After modifying the configuration file, you need to recompile in the workspace and source:

```
cd ~/yahboomcar_ws
colcon build && source install/setup.bash
```

3.2 Start and Test the Function

1. Open a new terminal and start the `audio_io` main program:

Run the following command to enable the microphone and wake-up program:

```
ros2 launch audio_io audio_io_yahboom.launch.py
```

2. Open a new terminal and start the `largemodel` main program:

Run the following command to enable voice interaction:

```
ros2 launch largemodel largemodel_control.launch.py
```

3. Test:

- **Wake-up:** Say to the microphone: "Hello, Digua."
- **Dialogue:** After the speaker responds, you can say: `what do you see?`
- **Observe logs:** In the terminal running the `launch` file, you should be able to see:
 1. The ASR node recognizes your question and prints it out.
 2. The `model_service` node receives the text, calls the LLM, and prints out the LLM's response.
- **Listen to the answer:** After a while, you should hear the answer from the speaker.